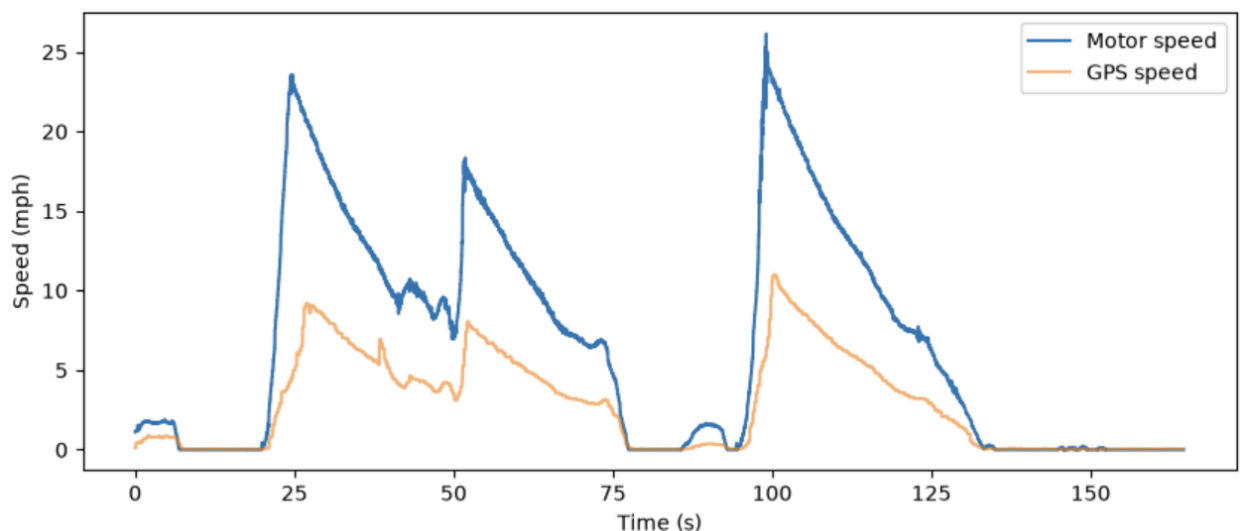


- Using parquet logic, forward fill and print all the values in a table format from the file. Provide a screenshot from your terminal display of what got printed.

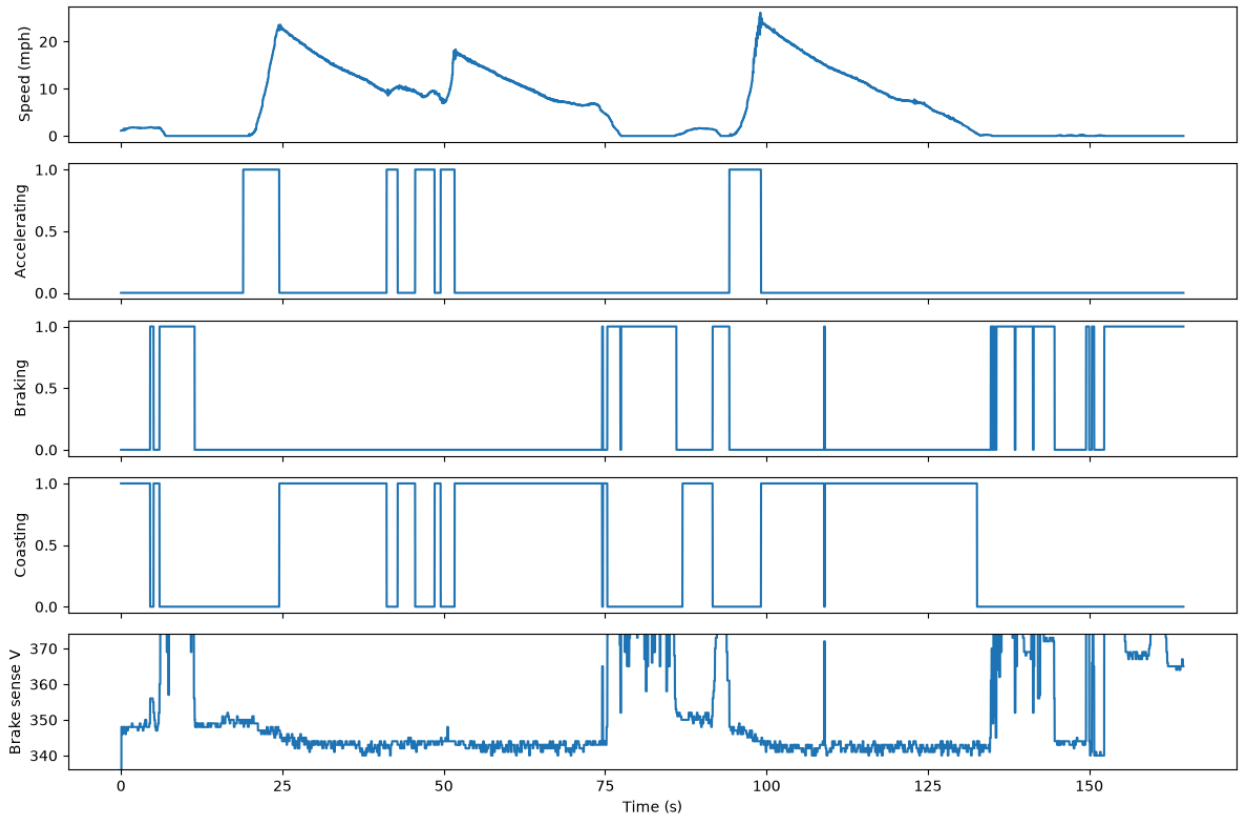
	Time	SME_THROTL_TorqueDemand	SME_THROTL_MaxSpeed	SME_THROTL_Forward	SME_THROTL_Reverse	SME_THROTL_UNUSED_BIT_1	SME_THROTL_PowerRea
0	0.000000	0.0	7500.0	1.0	0.0	False	0
1	0.011917	0.0	7500.0	1.0	0.0	False	0
2	0.023833	0.0	7500.0	1.0	0.0	False	0
3	0.035750	0.0	7500.0	1.0	0.0	False	0
4	0.047666	0.0	7500.0	1.0	0.0	False	0
5	0.059583	0.0	7500.0	1.0	0.0	False	0
6	0.071500	0.0	7500.0	1.0	0.0	False	0
7	0.083416	0.0	7500.0	1.0	0.0	False	0
8	0.095333	0.0	7500.0	1.0	0.0	False	0
9	0.107249	0.0	7500.0	1.0	0.0	False	0
10	0.119166	0.0	7500.0	1.0	0.0	False	0
11	0.131082	0.0	7500.0	1.0	0.0	False	0
12	0.142999	0.0	7500.0	1.0	0.0	False	0
13	0.154916	0.0	7500.0	1.0	0.0	False	0
14	0.166832	0.0	7500.0	1.0	0.0	False	0
15	0.178749	0.0	7500.0	1.0	0.0	False	0
16	0.190665	0.0	7500.0	1.0	0.0	False	0
17	0.202582	0.0	7500.0	1.0	0.0	False	0
18	0.214499	0.0	7500.0	1.0	0.0	False	0
19	0.226415	0.0	7500.0	1.0	0.0	False	0

Used ffill() to forward fill and remove missing values.

- Using matplotlib, graph the speed of the car(in MPH) vs time. Refer to the data_columns.csv to know which column means what. Use that graph to find the speed of the car at 10s. To find the speed of the car, use 'SME_TRQSPD_Speed.' It is measured in RPM which you will need to convert. Please refer to the physics section below for the explanation.



- Give a time frame for when the car is accelerating, braking, and coasting. Explain how you found these values out. Then plot those states over time on three different graphs



First thing I did was graph each of the 3 behaviors, and to do this I first needed to define what constitutes braking, coasting, and accelerating. Originally I was going to use the brake light indicator as an easy way to see if the car was braking or not, but found that it stayed on even when the car was accelerating, leading to this not being an ideal solution. Instead, I used "ETC_STATUS_BRAKE_SENSE_VOLTAGE", which was the level that the brake pedal was pressed. The issue here I ran into was that the brake pedal signal was noisy, and I had to find a minimum value that classified the pedal as either pressed or not pressed. I tested multiple values by setting the minimum as a part in the function (`df["ETC_STATUS_BRAKE_SENSE_VOLTAGE"] > x`), where `x` was the minimum. I decided upon 352, as this seemed the most accurate to the other graphs, and speed graph, and matched the spikes shown on the brake sense graph without including the tiny fluctuations.

For coasting, I set it to whenever the car is moving at $> 0.5\text{mph}$, and is not braking or accelerating. Finally for accelerating, I set it to whenever the "SME_THROTL_TorqueDemand" was greater than 0.

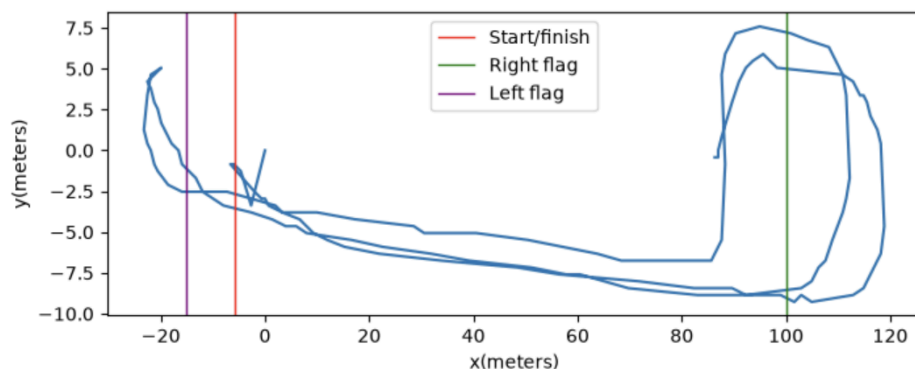
To find the intervals where each of these events occurred, I created 3 separate cells, each corresponding to a variable, and mapped whenever there was a change from 0 to 1, or vice versa, and logged if it was either switching on or off. As an example, 4.50 On, 5.04 Off, can be read as On on the interval (4.50, 5.04). I found the intervals as:

Braking	
t	event
4.50	ON
5.04	OFF
5.96	ON
11.40	OFF
74.54	ON
74.65	OFF
75.33	ON
77.36	OFF
77.48	ON
86.04	OFF
91.65	ON
94.24	OFF
108.92	ON
109.04	OFF
134.74	ON
134.98	OFF
135.15	ON
135.38	OFF
135.62	ON
138.45	OFF
138.57	ON
141.26	OFF
141.38	ON
144.63	OFF
149.51	ON
149.98	OFF
150.40	ON
150.76	OFF
152.32	ON

Coasting	
t	event
4.50	OFF
5.04	ON
5.96	OFF
24.51	ON
41.15	OFF
42.86	ON
45.56	OFF
48.58	ON
49.52	OFF
51.68	ON
74.54	OFF
74.65	ON
75.33	OFF
86.98	ON
91.65	OFF
99.14	ON
108.92	OFF
109.04	ON
132.61	OFF

Accelerating	
t	event
18.92	ON
24.51	OFF
41.15	ON
42.86	OFF
45.56	ON
48.58	OFF
49.52	ON
51.68	OFF
94.24	ON
99.14	OFF

4. Find out how many laps the car drove as well as the start and end time for each lap. The car has a GPS system you can utilize to visualize this. Provide a screenshot of your method along with an explanation of how you went about finding this.

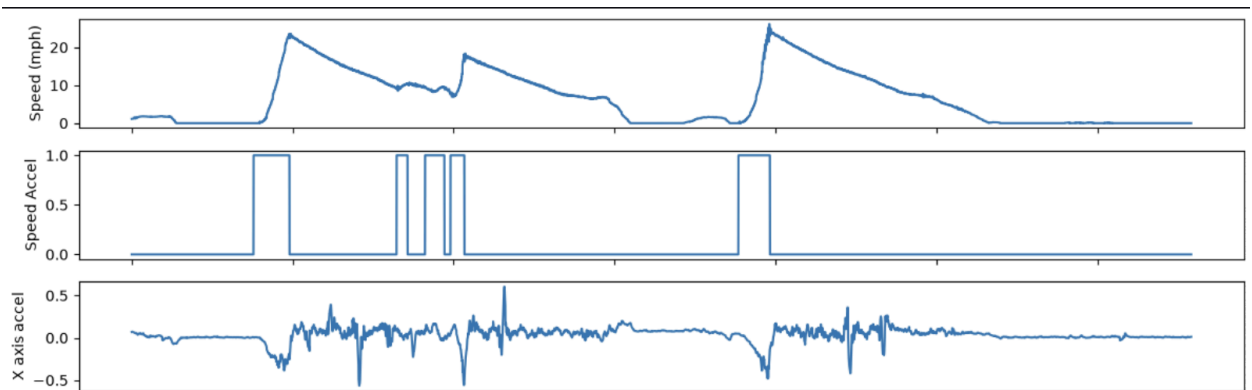


I first started by converting the lat/long coordinates into meters, as the lat/lon coordinates were harder to read. Then I created 3 conditions that the car needed to satisfy to count a lap, being that the car needed to cross $x = 100$, $x = -15$, and pass back through $x=0$. I got that total, the car completed 1 lap, and got about halfway through completing

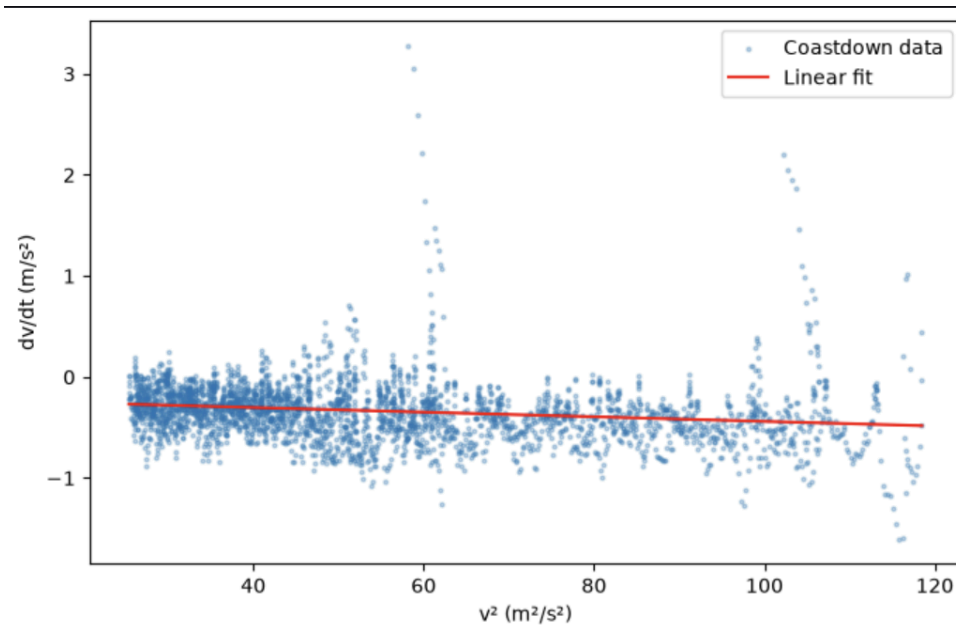
another. Finding the start time was easy, as you could use the acceleration intervals, and compare them with the visualized car path to see when it began racing, and this would show that the car takes a few seconds to get into position, then the start acceleration happens at 18.92. I had to play around with where the track actually began, but eventually I found at ($x = -5.685$) was the closest to the acceleration time while also staying in a location that the car passed over multiple times. I got that the lap time was approximately $99.9 - 18.92$, or 80.98 seconds.

5. Using the lap times found from part 4, determine the max speed, max acceleration, time spent accelerating, and time spent coasting.

Max speed for lap 1 was 26.1356 mph, Max accel was 17.9918 mph/s, found by smoothing and finding the gradient of speed. Time spent accelerating was 17.3777sec, and time spent coasting was 49.2432sec. I testing using X and Y accel from the parquet but the correlation between that data and the speed graph was not sufficient enough for me, and the acceleration through torque demand was more accurate.



- 6. From the coasting data (from all laps), remove anything below 5 m/s and anything below a second.**
- 7. Use that coast down data to determine vehicle drag + rolling resistance. Please refer below for the explanation of the physics.**



After isolating the coastdown data with intervals 1 second and speeds over 5m/s, plotting the remaining points and calculating the polyfit line gives us a slope of -0.002286, which from plugging into the drag area formula gives us 0.91m^2 . Then, using the intercept of -0.2108, we get that that the rolling resistance is 0.0237.

8. Write 3 or more paragraphs about what you learned about the car, and your takeaways from the data.

Much of this project introduced new concepts I had never delt with, such as matplotlib, vehicle physics, and dealing with large amounts of data-tables. I also found out many specifics about the car, with the first one being that the sensors within the car report at different rates, leading to the data table being full of gaps. This led me to learn about `ffill()`, which brings down the most recent data until it changes again, which is a handy function that I will most likely use many times in data science projects.

I also learned about signal noise, and how many of these signals are not exactly accurate all of the time. A good example of this is the brake sense variable, which had constant noise, and thus to get accurate data out of it, it took some checking and guessing to find the right values that are considered “on” and “off”. Same thing with the coastdown data, the data was not exactly a straight line, and thus it took some averaging to find the correct slope and intercept.

Some issues I faced were in regards to the final few problems, where because I am new to using some of these modules, I had to look through docs and use AI to assist in converting what I had intended into code. I also had to learn about how air drag, and wheel resistance were calculated, while also putting this into practice using advanced code I've not used before.

Lastly, one thing that I found cool about this is that the same techniques used in games like Mario Kart were able to be used here for identifying laps. In Mario Kart, markers are placed along the track that must be hit or else a lap does not count, as to prevent going back and forth over the starting line to quickly finish the race. I applied similar techniques here, where 3 markers were set up, one at the finish, and 2 others on

each side of the track, so that now when the car goes back and forth by the line, it isn't counted as multiple laps. But overall I found this project very fun and it challenged me in ways I haven't really delved into before, and I hope to join the data team to pursue this further.